

P.PORTO	ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Tipo de Prova Exame Escrito – Época Normal	Ano letivo 2025/2026	Data 15-06-2026
			Licenciatura em Engenharia em Informática	Hora 10:00
		Curso Licenciatura em Engenharia Informática/Licenciatura em Segurança Informática em Redes de Computadores	Ano letivo 2025/2026	Data 15-06-2026
		Unidade Curricular Paradigmas de Programação	Hora 10:00	Duração 2 horas

Observações

Não é permitida a consulta.

Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

(1,5 valores)

1 Explique detalhadamente as diferenças entre classes abstratas e interfaces em Java. Em que situações é mais adequado optar por uma classe abstrata e em que situações é preferível uma interface? Justifique a sua resposta e ilustre cada caso com um exemplo prático que evidencie as características distintivas de cada mecanismo.

(1,5 valores)

2 Descreva detalhadamente o modo como Java realiza a passagem de argumentos para os métodos, distinguindo o comportamento aplicado a tipos primitivos do comportamento aplicado a referências de objetos. Esclareça os equívocos frequentes associados a este mecanismo e fundamente a sua explicação com exemplos concretos que demonstrem os efeitos sobre os valores e os estados dos objetos.

(1,5 valores)

3 Explique o conceito de conversão de tipos (*casting*) no contexto da herança e do polimorfismo. Discuta os riscos associados a conversões incorretas e as situações em que cada tipo de conversão é apropriado. Ilustre com um exemplo prático que demonstre as conversões.

(1,5 valores)

4 Distinga os conceitos de identidade e de igualdade de objetos em Java, esclarecendo a diferença entre o operador `==` e o método `equals()`. Explique igualmente o papel do método `toString()`. Forneça um exemplo prático que demonstre a redefinição correta dos métodos `equals()` e `toString()` numa classe.

		Ano letivo 2025/2026	Data 15-06-2025
P.PORTO	ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Tipo de Prova Exame Escrito - Época Normal	Hora 10:00
		Curso Licenciatura em Engenharia Informática/Licenciatura em Segurança Informática em Redes de Computadores	
		Unidade Curricular Paradigmas de Programação	Duração 2 horas

Parte 2

1. Considere as seguintes interfaces `Vehicle` e `RefrigeratedVehicle`. A interface `Vehicle` descreve as operações possíveis que definem o contrato de um veículo utilizado na recolha de bens de uma instituição de ajuda humanitária. A interface `RefrigeratedVehicle` especializa `Vehicle` e acrescenta operações específicas para veículos refrigerados com restrições de distância.

```
public interface Vehicle {
    String getCode();
    ItemType getSupplyType();
    double getMaxCapacity();
}
```

```
public interface RefrigeratedVehicle extends Vehicle {
    double getMaxKilometers();
    boolean equals(Object obj);
}
```

(3 valores)

1a. Considere a interface `RefrigeratedVehicle` que representa um veículo refrigerado com um limite máximo de quilómetros. Implemente a interface numa classe denominada `RefrigeratedVehicleImpl`. O veículo deve possuir um **estado** (`ENABLED`, `DISABLED`) e deve ser inicializado como `ENABLED` por defeito. Para a implementação do método `equals`, considere que **duas** instâncias de `RefrigeratedVehicle` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`).

(2 valores)

1b. Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

2. Considere a existência de uma classe `StrategyImpl` que implemente a interface `Strategy`.

```
public interface Strategy {
    Route[] generate(IIInstitution inst, RouteValidator validator);
}
```

(4 valores)

2a. Implemente os seguintes métodos que podem ser utilizados para a geração da rota.

```
boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox);
```

- Este método deve devolver `true` caso exista a ocorrência de, pelo menos, um container cujo tipo seja igual ao do veículo e se a sua última medição registada tiver um valor superior a 80% da sua capacidade.

```
boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator);
```

- O método deve devolver `true` caso a `AidBox` seja adicionada à rota.
- Para adicionar a `AidBox` à rota, deve previamente validar efetuar uma validação com recurso ao método `validator.validate(Route route, AidBox aidbox)`.

P.PORTO	ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Tipo de Prova	Ano letivo	Data
		Exame Escrito - Época Normal	2025/2026	15-06-2026
		Curso	Licenciatura em Engenharia Informática/Licenciatura em Segurança Informática em Redes de Computadores	
		Unidade Curricular Paradigmas de Programação	Hora	Duração
			10 00	2 horas

- o Se validado, a `AidBox` deve ser adicionada à rota utilizando o método `void addAidBox(AidBox aidBox)` da classe `Route`
- o Caso a invocação do método `addAidBox` origine uma `RouteException`, o método `addAidBoxToRoute` deve retornar falso.

(5 valores)

2.b. Na classe `StrategyImpl`, implemente o método `generate`, gerando as rotas necessárias

Regras a considerar

- Para cada veículo devolvido pelo método `getVehicles()` da interface `Institution`, deve ser criada uma nova rota `Assuma` que só existe um veículo para cada tipo
- As `AidBoxes` existentes são as devolvidas pelo método `getAidBoxes()` da interface `Institution`.
- Deve utilizar os métodos desenvolvidos na alínea anterior. Se não os implementou anteriormente, assumo que os métodos já existem
- O array devolvido pelo método `generate` deve conter rotas com as `AidBoxes` (sem posições nulas ou rotas vazias)

Para além dos excertos de código fornecidos, considere o seguinte resumo dos métodos para a resolução do exercício

```
public class InstitutionImpl implements IInstitution {
    (...)
    private AidBox[] aidBoxes;
    private int numberOfAidBoxes;
    private Vehicle[] vehicles;
    private int numberOfVehicles;
    (...)

    @Override
    public int getTotalContainersByType(ItemType type) {
        (...)
    }
    public Vehicle[] getVehicles(){
        (...)
    }
    public AidBox[] getAidBoxes(){
        (...)
    }
}
```

```
public interface AidBox {
    Container[] getContainers();
    ...
}
```


P.PORTO ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Tipo de Prova Exame Escrito – Época Normal		Ano letivo 2025/2026	Data 15-06-2026
	Curso Licenciatura em Engenharia Informática/Licenciatura em Segurança Informática em Redes de Computadores			Hora 10.00
	Unidade Curricular Paradigmas de Programação			Duração 2 horas

```
public interface Container {
    ItemType getType();
    double getCapacity();
    Measurement getLastMeasurement();
    ...
}
```

```
public interface Measurement {
    double getValue();
    ...
}
```

```
public interface Route {
    void addAidBox(AidBox aidBox) throws RouteException;
    AidBox removeAidBox(AidBox aidBox) throws RouteException;
    AidBox[] getRoute();
}
```